

Revision Notes for Class 12 Computer Science Chapter 4 – Queue and Deque

Queue and Deque are important data structures in Python and have many applications in both daily life and computer science. A queue works on a First-In-First-Out (FIFO) principle, meaning the first element added will be the first to be removed. This is similar to a line of people at a ticket counter or cars waiting at a petrol pump.

In a queue, new elements are always inserted at the rear end while removal happens from the front. The rear is also called the tail, and the front is referred to as the head. This arrangement ensures fairness, much like waiting turns in real-life queues.

Queues are widely used in everyday scenarios as well as in computer systems. For example, managing print jobs, handling website server requests, and organizing tasks in operating systems all use queues to process data in a fair and orderly fashion. Real-life queues help us visualize the importance of this structure.

Key Operations of Queue

Queues allow five main actions—enqueue, dequeue, isEmpty, peek, and isFull. When we use “enqueue,” we add a new item to the rear. “Dequeue” removes an item from the front. Checking if the queue is empty or full avoids errors like underflow and overflow. The “peek” operation helps us see the front item without removing it.

- ENQUEUE: Add an element at the rear end.
- DEQUEUE: Remove an element from the front end.
- IS EMPTY: Check if the queue has elements.
- PEEK: Show the front element without removing it.
- IS FULL: Check if the queue has reached its maximum size.

Overflow occurs when we try to add more items than the queue's capacity allows, while underflow happens if we try to remove an item from an empty queue. These concepts are vital for writing safe and efficient programs.

Queue Implementation in Python

Python makes implementing queues straightforward using lists. The enqueue function uses the append method for adding at the rear, and dequeue uses pop(0) to remove from the front. Other helper functions like isEmpty, size, and peek help manage the queue easily.

- Appending to a list: Adds to rear.
- Removing from index 0: Removes from front.
- Consistent use of ends is key for correct results.

In practical examples, like simulating a bank queue, you ask for person codes to insert or remove them from the queue and display the number of people remaining. If you try to dequeue an empty queue, it's important to handle this without causing an error. Students learn by experimenting with these small programs, reinforcing how queues behave in real scenarios.

Applications of Queue

Queues appear in ticket booking systems, printers, CPU task management, and even customer care call centers. In each situation, tasks or requests are processed in the same order they come, ensuring fairness.

- Waiting lists for train tickets are managed as queues.
- Printing documents from multiple computers is controlled by a print queue.
- Web servers queue requests to avoid overload.

Computer queues provide fairness and efficiency, especially when many users or tasks need attention at once. For instance, schools often use queues for students at the assembly or for distributing resources.

Introduction to Deque

A deque, short for double-ended queue and pronounced “deck”, is a flexible data structure where elements can be added or removed from both the front and rear. Unlike queues, which restrict removal and addition to specific ends, deques allow both.

This feature lets deques act as either stacks (LIFO) or queues (FIFO) by controlling which end is used for insertion and deletion. In browsers, for example, the history function uses a deque—closing many tabs removes the oldest ones, and new tabs appear at the “rear”.

Deque Operations

The basic operations in a deque mirror those in a queue but are open to both ends:

- INSERTFRONT: Add an element to the front.
- INSERTREAR: Add an element to the rear.
- DELETIONFRONT: Remove from the front end.
- DELETIONREAR: Remove from the rear end.
- Other support: isEmpty, peek, size, getFront, getRear.

When both insertion and removal occur at the same end, the deque acts as a stack. When they’re at opposite ends, it behaves like a queue. This makes deques a fundamental building block for many practical algorithms.

Checking Palindromes with Deque

A key application is checking for palindromes using a deque. Each letter is inserted from one end, and letters are removed from both ends for comparison. If each matched pair is equal, the string is a palindrome. This method is systematic and makes use of deque operations smartly.

Deque Implementation in Python

Deques can also be created using Python lists. Functions control each operation with `insert(0, element)` for front insertion and `append(element)` for rear. Removal uses `pop(0)` and `pop()` for front and rear respectively. Checking if the deque is empty before operations prevents underflow errors.

- `insertFront/deletionFront`: Involve the start of the list.
- `insertRear/deletionRear`: Involve the end of the list.
- `getFront/getRear`: Look at front or rear without removing elements.

Sample programs illustrate how a deque can behave like a normal queue or a stack, depending on user choices for insertion or deletion. Outputs show underflow tries (attempts to remove from an empty deque), which should be handled gracefully.

Summary and Differences

Queues are best when you need a strict order, like managing tickets, while deques add flexibility, catering to more use-cases including undo/redo features and palindrome checks. They share operations but differ fundamentally in access—a queue is single-end addition and removal; deque is double-ended.

- Queue: FIFO, single direction.
- Deque: Can be FIFO or LIFO, two-ended operations.
- Stacks: LIFO, single end.

Being comfortable with these data structures is essential for students preparing for board exams and for programming in general. Attempting the exercises given in the chapter, like comparing stack vs queue or implementing palindrome check, will further solidify these concepts.

Understanding when and where to use a queue or a deque comes with practice, as each structure offers unique advantages in code as well as in real life. Always check for underflow and overflow, and choose the right data structure based on your problem.